

MiroFish-Offline

Change Report

Custom enhancements layered on top of the baseline OASIS simulation engine

Generated: 24 April 2026, 21:16

Author: Dolores · hariharan.m@mu-sigma.com

1. Executive Summary

MiroFish-Offline is a multi-agent social simulation engine built on the OASIS/CAMEL-AI framework with a Neo4j knowledge graph backend and a Vue 3 frontend. The baseline repository ran with a local Ollama container and offered no runtime model control, no visibility into LLM calls, no bag-ontology inspection, and no way to pause or reorder agents mid-simulation.

This report documents every change made to introduce four capabilities requested by the user, plus the network connectivity fix:

- **Model Chooser** — pick any of the 59 models on the remote Ollama server (172.25.1.70:11434) from the UI header, switch server URLs, all without restarting.
- **Transparent Agentic Conversation** — every LLM prompt and response is logged in real-time, visible in an expandable side panel during simulation.
- **Bag Ontology Visibility** — entity types and edge types from the Neo4j knowledge graph are surfaced in a live side panel during simulation.
- **Pause & Rearrange** — pause a running simulation, reorder agents, adjust influence/activity sliders, inject one-off events, then resume.
- **Network Fix** — API calls now use relative paths proxied by Vite, so the app works from any IP/hostname, not just localhost.

2. File Change Summary

File / Component	Status	Impact
.env	NEW	Remote Ollama + port config
backend/app/config.py	MODIFIED	Runtime model & server override
backend/app/utils/llm_client.py	MODIFIED	Global LLM call logging
backend/app/api/settings.py	NEW	7 new API endpoints
backend/app/api/simulation.py	MODIFIED	6 new simulation control endpoints
backend/app/api/__init__.py	MODIFIED	Blueprint registration
backend/app/__init__.py	MODIFIED	Blueprint registration
frontend/src/api/settings.js	NEW	API client for all new endpoints
frontend/src/components/ModelChooser.vue	NEW	Live model selector UI
frontend/src/views/MainView.vue	MODIFIED	ModelChooser added to header
frontend/src/components/Step3Simulation.vue	MODIFIED	Pause/resume, ontology, LLM log UI
frontend/vite.config.js	MODIFIED	Proxy port 5001→5010
frontend/src/api/index.js	MODIFIED	Relative baseURL (network fix)
docker-compose.yml	MODIFIED	Removed local ollama service

3. Infrastructure & Configuration

3.1 .env (NEW FILE)

The repository originally shipped no .env — environment was baked into docker-compose. A new .env was created for bare-metal execution pointing all services at the remote Ollama server instead of a local container:

-	# no .env existed
+	LLM_BASE_URL=http://172.25.1.70:11434/v1
+	LLM_MODEL_NAME=qwen2.5:32b
+	OLLAMA_NUM_CTX=8192
+	NEO4J_URI=bolt://localhost:7687
+	FLASK_PORT=5010 # avoids collision with :5001
+	OPENAI_API_BASE_URL=http://172.25.1.70:11434/v1

3.2 docker-compose.yml (MODIFIED)

The local ollama service and its depends_on reference were removed. All LLM traffic goes to the remote server. Only the mirofish app and neo4j services remain.

```
- ollama:
- image: ollama/ollama
- volumes: [ollama_data:/root/.ollama]
- depends_on: [ollama] # in mirofish service
```

4. Backend Changes

4.1 backend/app/config.py (MODIFIED)

Added class-level runtime override slots so model and server URL can be switched at runtime via the API — no process restart required. The active URL/model is always resolved through a getter that prefers the runtime override:

+	from typing import Optional
+	_runtime_model: Optional[str] = None
+	_runtime_base_url: Optional[str] = None
+	get_runtime_model() / set_runtime_model(model)
+	get_active_base_url() / set_runtime_base_url(url)

4.2 backend/app/utils/llm_client.py (MODIFIED)

Three additions: (1) the constructor now resolves base_url and model through Config's runtime getters; (2) a global in-memory ring buffer (deque, max 200 entries) captures every LLM call; (3) an optional per-simulation JSONL file is written alongside each call for persistent retrieval.

+	_global_llm_log: deque = deque(maxlen=200)
+	get_global_llm_log() → List[Dict]
~	base_url = base_url or Config.LLM_BASE_URL base_url = base_url or Config.get_active_base_url()
~	model = model or Config.LLM_MODEL_NAME model = model or Config.get_runtime_model() or Config.LLM_MODEL_NAME
+	call_tag, simulation_log_path params on __init__
+	_log_call() → appends entry to deque + optional JSONL file
+	chat() calls self._log_call() after every completion

4.3 backend/app/api/settings.py (NEW FILE)

New Flask Blueprint registered at `/api/settings`. Provides full runtime control over the LLM configuration and exposes the global call log:

Method	Endpoint	Purpose
GET	<code>/api/settings</code>	Current model, base_url, override flags
GET	<code>/api/settings/models</code>	Live model list from Ollama (<code>/api/tags</code>). Optional <code>?url=</code> param to probe a different server without saving
POST	<code>/api/settings/model</code>	Switch active model at runtime. Body: <code>{model}</code>
DELETE	<code>/api/settings/model</code>	Reset model to .env default
POST	<code>/api/settings/server</code>	Switch Ollama server URL. Probes server, returns live model list
DELETE	<code>/api/settings/server</code>	Revert server URL to .env default
GET	<code>/api/settings/llm-calls</code>	Global LLM transparency log (newest-first, <code>?limit=N</code> , <code>?tag=X</code>)

4.4 backend/app/api/simulation.py (MODIFIED)

Six new endpoints appended to the existing simulation blueprint. These power the pause/resume, agent rearrangement, event injection, ontology, and per-simulation LLM call log features:

Method	Endpoint	Purpose
GET	<code>//ontology</code>	Read entity types + edge types from Neo4j storage
POST	<code>/pause</code>	Stop simulation subprocess, set status=PAUSED
PUT	<code>//agents</code>	Patch agent configs and reorder in <code>simulation_config.json</code>
POST	<code>/resume</code>	Mark simulation READY, restart subprocess
POST	<code>//inject-event</code>	Prepend a post/event to <code>event_config.initial_posts</code>
GET	<code>//llm-calls</code>	Stream last N entries from <code>llm_calls.jsonl</code> for this simulation

5. Frontend Changes

5.1 frontend/src/api/settings.js (NEW FILE)

Thin Axios wrapper functions for every new backend endpoint:

<code>getSettings()</code>	→ GET <code>/api/settings</code>
<code>listModels(url?)</code>	→ GET <code>/api/settings/models</code>
<code>setModel(model)</code>	→ POST <code>/api/settings/model</code>
<code>resetModel()</code>	→ DELETE <code>/api/settings/model</code>
<code>setServer(url)</code>	→ POST <code>/api/settings/server</code>
<code>resetServer()</code>	→ DELETE <code>/api/settings/server</code>
<code>getSimulationOntology(simId)</code>	→ GET <code>/api/simulation/:id/ontology</code>
<code>pauseSimulation(simId)</code>	→ POST <code>/api/simulation/pause</code>
<code>resumeSimulation(simId, opts)</code>	→ POST <code>/api/simulation/resume</code>
<code>updateAgentConfigs(simId, agents, order)</code>	→ PUT <code>/api/simulation/:id/agents</code>
<code>injectEvent(simId, content, author)</code>	→ POST <code>/api/simulation/:id/inject-event</code>
<code>getSimulationLlmCalls(simId, limit)</code>	→ GET <code>/api/simulation/:id/llm-calls</code>

5.2 frontend/src/components/ModelChooser.vue (NEW FILE)

A compact dropdown button added to the app header. Shows the current model name and a green/red health dot indicating whether the Ollama server is reachable.

- Clicking the button opens a popover with a server URL input field.
- Hitting *Connect* calls `POST /api/settings/server`, probes the server, and loads the live model list.
- All 59 remote models are shown in a scrollable list; clicking one calls `POST /api/settings/model`.
- A custom model name text field allows typing any model name directly.
- State is synced from `GET /api/settings` on open.

5.3 frontend/src/views/MainView.vue (MODIFIED)

One-line change — `<ModelChooser />` imported and placed in the header-right slot.

```
+ import ModelChooser from
+   '../components/ModelChooser.vue'
```

5.4 frontend/src/components/Step3Simulation.vue (MODIFIED)

The simulation view received the largest frontend change. Four new interactive capabilities were wired into the existing control bar:

Pause & Rearrange

- A *Pause & Rearrange* button calls `POST /api/simulation/pause` and opens a full-screen overlay modal.
- The overlay lists every agent with influence (slider) and activity (slider) controls and up/down arrow buttons to reorder the agent execution sequence.
- An *Inject Event* row lets the user type a message that gets prepended to the simulation's event queue immediately.
- *Save & Resume* calls `PUT /api/simulation/:id/agents` to persist the new config, then `POST /api/simulation/resume` to restart the subprocess.

Bag Ontology Panel

- Antenna-icon button toggles a right-side panel.
- Panel calls `GET /api/simulation/:id/ontology` and renders entity types (nodes) and edge types (relationships) from the Neo4j knowledge graph.
- Entity types shown as teal badges; edge types shown as amber badges.

LLM Call Log Panel

- Chat-bubble-icon button toggles a right-side panel that auto-polls every 5 s.
- Each entry shows model name, tag, elapsed time, and a one-line preview.
- Clicking an entry expands the full prompt messages and model response inline.
- Calls `GET /api/simulation/:id/llm-calls` for per-simulation history.

5.5 frontend/vite.config.js (MODIFIED)

Proxy target updated from the old port to the new one:

```
~ target: 'http://localhost:5001' target: 'http://localhost:5010'
```

5.6 frontend/src/api/index.js (MODIFIED) — Network Fix

The original code hard-coded `http://localhost:5001` as the Axios base URL. When the app is opened from another machine (e.g. `http://172.25.2.181:3010`), the browser tries to reach `localhost:5001` on the *user's* machine — which fails.

Fix: base URL set to an empty string so all `/api/*` requests are relative to the page origin. Vite's dev-server proxy intercepts them and forwards to the backend on `localhost:5010` server-side:

```
~ baseUrl: import.meta.env.VITE_API_BASE_URL  baseUrl: '' // relative - Vite proxies
|| 'http://localhost:5001'                    /api/* → :5010
```

6. How to Run

All services run bare-metal (no Docker rebuild needed). Neo4j runs in a single Docker container.

```
# 1. Start Neo4j
docker run -d --name mirofish-neo4j \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/mirofish \
  neo4j:5.18-community

# 2. Start Flask backend (from repo root)
python3 backend/run.py          # reads .env → port 5010

# 3. Start Vue frontend (from frontend/)
npm install && npm run dev -- --port 3010 --host 0.0.0.0

# App:      http://<host>:3010
# API:      http://<host>:5010
# Neo4j:    http://<host>:7474
```

7. Architecture Overview

